Contributor license agreement

By submitting code as an individual you agree to the individual contributor license agreement. By submitting code as an entity you agree to the corporate contributor license agreement.

This notice should stay as the first item in the CONTRIBUTING.MD file.

Table of Contents *generated with DocToc*

Contribute to GitLab

Thank you for your interest in contributing to GitLab. This guide details how to contribute to GitLab in a way that is efficient for everyone.

GitLab comes into two flavors, GitLab Community Edition (CE) our free and open source edition, and GitLab Enterprise Edition (EE) which is our commercial edition. Throughout this guide you will see references to CE and EE for

abbreviation.

If you have read this guide and want to know how the GitLab core team operates please see the GitLab contributing process.

- GitLab Inc engineers should refer to the engineering workflow document

Security vulnerability disclosure

Please report suspected security vulnerabilities in private to `support@gitlab.com`, also see the disclosure section on the GitLab.com website. Please do **NOT** create publicly viewable issues for suspected security vulnerabilities.

Closing policy for issues and merge requests

GitLab is a popular open source project and the capacity to deal with issues and merge requests is limited. Out of respect for our volunteers, issues and merge requests not in line with the guidelines listed in this document may be closed without notice.

Please treat our volunteers with courtesy and respect, it will go a long way towards getting your issue resolved.

Issues and merge requests should be in English and contain appropriate language for audiences of all ages.

If a contributor is no longer actively working on a submitted merge request we can decide that the merge request will be finished by one of our Merge request coaches or close the merge request. We make this decision based on how important the change is for our product vision. If a Merge request coach is going to finish the merge request we assign the ~"coach will finish" label.

Helping others

Please help other GitLab users when you can. The channels people will reach out on can be found on the getting help page.

Sign up for the mailing list, answer GitLab questions on StackOverflow or respond in the IRC channel. You can also sign up on CodeTriage to help with the remaining issues on the GitHub issue tracker.

I want to contribute!

If you want to contribute to GitLab, but are not sure where to start, look for issues with the label `Accepting Merge Requests` and weight < 5. These issues will be of reasonable size and challenge, for anyone to start contributing to GitLab.

Workflow labels

To allow for asynchronous issue handling, we use milestones and labels. Leads and product managers handle most of the scheduling into milestones. Labelling is a task for everyone.

Most issues will have labels for at least one of the following:

- Type: ~"feature proposal", ~bug, ~customer, etc.
- Subject: ~wiki, ~"container registry", ~ldap, ~api, etc.
- Team: ~CI, ~Discussion, ~Edge, ~Frontend, ~Platform, etc.
- Priority: ~Deliverable, ~Stretch

All labels, their meaning and priority are defined on the labels page.

If you come across an issue that has none of these, and you're allowed to set labels, you can *always* add the team and type, and often also the subject.

Type labels (~"feature proposal", ~bug, ~customer, etc.)

Type labels are very important. They define what kind of issue this is. Every issue should have one or more.

Examples of type labels are ~"feature proposal", ~bug, ~customer, ~security, and ~"direction".

A number of type labels have a priority assigned to them, which automatically makes them float to the top, depending on their importance.

Type labels are always lowercase, and can have any color, besides blue (which is already reserved for subject labels).

The descriptions on the labels page explain what falls under each type label.

Subject labels (~wiki, ~"container registry", ~ldap, ~api, etc.)

Subject labels are labels that define what area or feature of GitLab this issue hits. They are not always necessary, but very convenient.

If you are an expert in a particular area, it makes it easier to find issues to work on. You can also subscribe to those labels to receive an email each time an issue is labelled with a subject label corresponding to your expertise.

Examples of subject labels are ~wiki, ~"container registry", ~ldap, ~api, ~issues, ~"merge requests", ~labels, and ~"container registry".

Subject labels are always all-lowercase.

Team labels (~CI, ~Discussion, ~Edge, ~Platform, etc.)

Team labels specify what team is responsible for this issue. Assigning a team label makes sure issues get the attention of the appropriate people.

The current team labels are ~Build, ~CI, ~Discussion, ~Documentation, ~Edge, ~Gitaly, ~Platform, ~Prometheus, ~Release, and ~"UX".

The descriptions on the labels page explain what falls under the responsibility of each team.

Within those team labels, we also have the ~backend and ~frontend labels to indicate if an issue needs backend work, frontend work, or both.

Team labels are always capitalized so that they show up as the first label for any issue.

Priority labels (~Deliverable and ~Stretch)

Priority labels help us clearly communicate expectations of the work for the release. There are two levels of priority labels:

- ~Deliverable: Issues that are expected to be delivered in the current milestone.
- ~Stretch: Issues that are a stretch goal for delivering in the current milestone. If these issues are not done in the current release, they will strongly be considered for the next release.

Label for community contributors (~"Accepting Merge Requests")

Issues that are beneficial to our users, 'nice to haves', that we currently do not have the capacity for or want to give the priority to, are labeled as ~"Accepting Merge Requests", so the community can make a contribution.

Community contributors can submit merge requests for any issue they want, but the ~"Accepting Merge Requests" label has a special meaning. It points to changes that:

1. We already agreed on,
2. Are well-defined,
3. Are likely to get accepted by a maintainer.

We want to avoid a situation when a contributor picks an ~"Accepting Merge Requests" issue and then their merge request gets closed, because we realize that it does not fit our vision, or we want to solve it in a different way.

We add the ~"Accepting Merge Requests" label to:

- Low priority ~bug issues (i.e. we do not add it to the bugs that we want to solve in the ~"Next Patch Release")
- Small ~"feature proposal" that do not need ~UX / ~"Product work", or for which the ~UX / ~"Product work" is already done
- Small ~"technical debt" issues

After adding the ~"Accepting Merge Requests" label, we try to estimate the weight of the issue. We use issue weight to let contributors know how difficult

the issue is. Additionally:

- We advertise ~"Accepting Merge Requests" issues with weight < 5 as suitable for people that have never contributed to GitLab before on the Up For Grabs campaign
- We encourage people that have never contributed to any open source project to look for ~"Accepting Merge Requests" issues with a weight of 1

Implement design & UI elements

Please see the UX Guide for GitLab.

Issue tracker

To get support for your particular problem please use the getting help channels.

The GitLab CE issue tracker on GitLab.com is for bugs concerning the latest GitLab release and feature proposals.

When submitting an issue please conform to the issue submission guidelines listed below. Not all issues will be addressed and your issue is more likely to be addressed if you submit a merge request which partially or fully solves the issue.

If you're unsure where to post, post to the mailing list or Stack Overflow first. There are a lot of helpful GitLab users there who may be able to help you quickly. If your particular issue turns out to be a bug, it will find its way from there.

If it happens that you know the solution to an existing bug, please first open the issue in order to keep track of it and then open the relevant merge request that potentially fixes it.

Issue triaging

Our issue triage policies are described in our handbook. You are very welcome to help the GitLab team triage issues. We also organize issue bash events once every quarter.

The most important thing is making sure valid issues receive feedback from the development team. Therefore the priority is mentioning developers that can help on those issues. Please select someone with relevant experience from the GitLab team. If there is nobody mentioned with that expertise look in the commit history for the affected files to find someone.

Feature proposals

To create a feature proposal for CE, open an issue on the issue tracker of CE.

For feature proposals for EE, open an issue on the issue tracker of EE.

In order to help track the feature proposals, we have created a `feature proposal` label. For the time being, users that are not members of the project cannot add labels. You can instead ask one of the core team members to add the label `feature proposal` to the issue or add the following code snippet right after your description in a new line: `~"feature proposal"`.

Please keep feature proposals as small and simple as possible, complex ones might be edited to make them small and simple.

Please submit Feature Proposals using the 'Feature Proposal' issue template provided on the issue tracker.

For changes in the interface, it can be helpful to create a mockup first. If you want to create something yourself, consider opening an issue first to discuss whether it is interesting to include this in GitLab.

Issue tracker guidelines

Search the issue tracker for similar entries before submitting your own, there's a good chance somebody else had the same issue or feature proposal. Show your support with an award emoji and/or join the discussion.

Please submit bugs using the 'Bug' issue template provided on the issue tracker. The text in the parenthesis is there to help you with what to include. Omit it when submitting the actual issue. You can copy-paste it and then edit as you see fit.

Issue weight

Issue weight allows us to get an idea of the amount of work required to solve one or multiple issues. This makes it possible to schedule work more accurately.

You are encouraged to set the weight of any issue. Following the guidelines below will make it easy to manage this, without unnecessary overhead.

1. Set weight for any issue at the earliest possible convenience
2. If you don't agree with a set weight, discuss with other developers until consensus is reached about the weight
3. Issue weights are an abstract measurement of complexity of the issue. Do not relate issue weight directly to time. This is called anchoring and something you want to avoid.
4. Something that has a weight of 1 (or no weight) is really small and simple. Something that is 9 is rewriting a large fundamental part of GitLab, which might lead to many hard problems to solve. Changing some text in GitLab is probably 1, adding a new Git Hook maybe 4 or 5, big features 7-9.
5. If something is very large, it should probably be split up in multiple issues or chunks. You can simply not set the weight of a parent issue and set weights to children issues.

Regression issues

Every monthly release has a corresponding issue on the CE issue tracker to keep track of functionality broken by that release and any fixes that need to be included in a patch release (see 8.3 Regressions as an example).

As outlined in the issue description, the intended workflow is to post one note with a reference to an issue describing the regression, and then to update that note with a reference to the merge request that fixes it as it becomes available.

If you're a contributor who doesn't have the required permissions to update other users' notes, please post a new note with a reference to both the issue and the merge request.

The release manager will update the notes in the regression issue as fixes are addressed.

Technical debt

In order to track things that can be improved in GitLab's codebase, we created the ~"technical debt" label in GitLab's issue tracker.

This label should be added to issues that describe things that can be improved, shortcuts that have been taken, code that needs refactoring, features that need additional attention, and all other things that have been left behind due to high velocity of development.

Everyone can create an issue, though you may need to ask for adding a specific label, if you do not have permissions to do it by yourself. Additional labels can be combined with the `technical debt` label, to make it easier to schedule the improvements for a release.

Issues tagged with the `technical debt` label have the same priority like issues that describe a new feature to be introduced in GitLab, and should be scheduled for a release by the appropriate person.

Make sure to mention the merge request that the `technical debt` issue is associated with in the description of the issue.

Stewardship

For issues related to the open source stewardship of GitLab, there is the ~"stewardship" label.

This label is to be used for issues in which the stewardship of GitLab is a topic of discussion. For instance if GitLab Inc. is planning to remove features from GitLab CE to make exclusive in GitLab EE, related issues would be labelled with ~"stewardship".

A recent example of this was the issue for bringing the time tracking API to GitLab CE.

Merge requests

We welcome merge requests with fixes and improvements to GitLab code, tests, and/or documentation. The issues that are specifically suitable for community contributions are listed with the label `Accepting Merge Requests` on our issue tracker for CE and EE, but you are free to contribute to any other issue you want.

Please note that if an issue is marked for the current milestone either before or while you are working on it, a team member may take over the merge request in order to ensure the work is finished before the release date.

If you want to add a new feature that is not labeled it is best to first create a feedback issue (if there isn't one already) and leave a comment asking for it to be marked as `Accepting Merge Requests`. Please include screenshots or wireframes if the feature will also change the UI.

Merge requests should be opened at GitLab.com.

If you are new to GitLab development (or web development in general), see the I want to contribute! section to get you started with some potentially easy issues.

To start with GitLab development download the GitLab Development Kit and see the Development section for some guidelines.

Merge request guidelines

If you can, please submit a merge request with the fix or improvements including tests. If you don't know how to fix the issue but can write a test that exposes the issue we will accept that as well. In general bug fixes that include a regression test are merged quickly while new features without proper tests are least likely to receive timely feedback. The workflow to make a merge request is as follows:

1. Fork the project into your personal space on GitLab.com
2. Create a feature branch, branch away from `master`
3. Write tests and code
4. Generate a changelog entry with `bin/changelog`
5. If you are writing documentation, make sure to follow the documentation styleguide
6. If you have multiple commits please combine them into a few logically organized commits by squashing them
7. Push the commit(s) to your fork
8. Submit a merge request (MR) to the `master` branch
9. Your merge request needs at least 1 approval but feel free to require more. For instance if you're touching backend and frontend code, it's a good idea to require 2 approvals: 1 from a backend maintainer and 1 from a frontend maintainer

10. You don't have to select any approvers, but you can if you really want specific people to approve your merge request
11. The MR title should describe the change you want to make
12. The MR description should give a motive for your change and the method you used to achieve it.
13. If you are contributing code, fill in the template already provided in the "Description" field.
14. If you are contributing documentation, choose `Documentation` from the "Choose a template" menu and fill in the template.
15. Mention the issue(s) your merge request solves, using the `Solves #XXX` or `Closes #XXX` syntax to auto-close the issue(s) once the merge request will be merged.
16. If you're allowed to, set a relevant milestone and labels
17. If the MR changes the UI it should include *Before* and *After* screenshots
18. If the MR changes CSS classes please include the list of affected pages, `grep css-class ./app -R`
19. Be prepared to answer questions and incorporate feedback even if requests for this arrive weeks or months after your MR submission
20. If a discussion has been addressed, select the "Resolve discussion" button beneath it to mark it resolved.
21. If your MR touches code that executes shell commands, reads or opens files or handles paths to files on disk, make sure it adheres to the shell command guidelines
22. If your code creates new files on disk please read the shared files guidelines.
23. When writing commit messages please follow these guidelines.
24. If your merge request adds one or more migrations, make sure to execute all migrations on a fresh database before the MR is reviewed. If the review leads to large changes in the MR, do this again once the review is complete.
25. For more complex migrations, write tests.
26. Merge requests **must** adhere to the merge request performance guidelines.
27. For tests that use Capybara or PhantomJS, see this article on how to write reliable asynchronous tests.

Please keep the change in a single MR **as small as possible**. If you want to contribute a large feature think very hard what the minimum viable change is. Can you split the functionality? Can you only submit the backend/API code? Can you start with a very simple UI? Can you do part of the refactor? The increased reviewability of small MRs that leads to higher code quality is more important to us than having a minimal commit log. The smaller an MR is the more likely it is it will be merged (quickly). After that you can send more MRs to enhance it. The 'How to get faster PR reviews' document of Kubernetes also has some great points regarding this.

For examples of feedback on merge requests please look at already closed merge requests. If you would like quick feedback on your merge request feel free to mention someone from the core team or one of the Merge request coaches. Please ensure that your merge request meets the contribution acceptance criteria.

When having your code reviewed and when reviewing merge requests please take the code review guidelines into account.

Contribution acceptance criteria

1. The change is as small as possible
2. Include proper tests and make all tests pass (unless it contains a test exposing a bug in existing code). Every new class should have corresponding unit tests, even if the class is exercised at a higher level, such as a feature test.
3. If you suspect a failing CI build is unrelated to your contribution, you may try and restart the failing CI job or ask a developer to fix the aforementioned failing test
4. Your MR initially contains a single commit (please use `git rebase -i` to squash commits)
5. Your changes can merge without problems (if not please rebase if you're the only one working on your feature branch, otherwise, merge `master`)
6. Does not break any existing functionality
7. Fixes one specific issue or implements one specific feature (do not combine things, send separate merge requests if needed)
8. Migrations should do only one thing (e.g., either create a table, move data to a new table or remove an old table) to aid retrying on failure
9. Keeps the GitLab code base clean and well structured
10. Contains functionality we think other users will benefit from too
11. Doesn't add configuration options or settings options since they complicate making and testing future changes
12. Changes do not adversely degrade performance.
    - Avoid repeated polling of endpoints that require a significant amount of overhead
    - Check for N+1 queries via the SQL log or `QueryRecorder`
    - Avoid repeated access of filesystem
13. If you need polling to support real-time features, please use polling with ETag caching.
14. Changes after submitting the merge request should be in separate commits (no squashing).
15. It conforms to the style guides and the following:
    - If your change touches a line that does not follow the style, modify the entire line to follow it. This prevents linting tools from generating warnings.
    - Don't touch neighbouring lines. As an exception, automatic mass refactoring modifications may leave style non-compliant.
16. If the merge request adds any new libraries (gems, JavaScript libraries, etc.), they should conform to our Licensing guidelines. See the instructions in that document for help if your MR fails the "license-finder" test with a "Dependencies that need approval" error.

Definition of done

If you contribute to GitLab please know that changes involve more than just code. We have the following definition of done. Please ensure you support the feature you contribute through all of these steps.

1. Description explaining the relevancy (see following item)
2. Working and clean code that is commented where needed
3. Unit and system tests that pass on the CI server
4. Performance/scalability implications have been considered, addressed, and tested
5. Documented in the `/doc` directory
6. Changelog entry added, if necessary
7. Reviewed and any concerns are addressed
8. Merged by a project maintainer
9. Added to the release blog article, if relevant
10. Added to the website, if relevant
11. Community questions answered
12. Answers to questions radiated (in docs/wiki/support etc.)

If you add a dependency in GitLab (such as an operating system package) please consider updating the following and note the applicability of each in your merge request:

1. Note the addition in the release blog post (create one if it doesn't exist yet) https://gitlab.com/gitlab-com/www-gitlab-com/merge_requests/
2. Upgrade guide, for example https://gitlab.com/gitlab-org/gitlab-ce/blob/master/doc/update/7.5-to-7.6.md
3. Upgrader https://gitlab.com/gitlab-org/gitlab-ce/blob/master/doc/update/upgrader.md#2-run-gitlab-upgrade-tool
4. Installation guide https://gitlab.com/gitlab-org/gitlab-ce/blob/master/doc/install/installation.md#1-packages-dependencies
5. GitLab Development Kit https://gitlab.com/gitlab-org/gitlab-development-kit
6. Test suite https://gitlab.com/gitlab-org/gitlab-ce/blob/master/scripts/prepare_build.sh
7. Omnibus package creator https://gitlab.com/gitlab-org/omnibus-gitlab

Style guides

1. Ruby. Important sections include Source Code Layout and Naming. Use:
   - multi-line method chaining style **Option A**: dot `.` on the second line
   - string literal quoting style **Option A**: single quoted by default
2. Rails
3. Newlines styleguide
4. Testing
5. JavaScript styleguide
6. SCSS styleguide
7. Shell commands created by GitLab contributors to enhance security

8. Database Migrations
9. Markdown
10. Documentation styleguide
11. Interface text should be written subjectively instead of objectively. It should be the GitLab core team addressing a person. It should be written in present time and never use past tense (has been/was). For example instead of *prohibited this user from being saved due to the following errors:* the text should be *sorry, we could not create your account because:*

This is also the style used by linting tools such as RuboCop, PullReview and Hound CI.

Code of conduct

As contributors and maintainers of this project, we pledge to respect all people who contribute through reporting issues, posting feature requests, updating documentation, submitting pull requests or patches, and other activities.

We are committed to making participation in this project a harassment-free experience for everyone, regardless of level of experience, gender, gender identity and expression, sexual orientation, disability, personal appearance, body size, race, ethnicity, age, or religion.

Examples of unacceptable behavior by participants include the use of sexual language or imagery, derogatory comments or personal attacks, trolling, public or private harassment, insults, or other unprofessional conduct.

Project maintainers have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that are not aligned to this Code of Conduct. Project maintainers who do not follow the Code of Conduct may be removed from the project team.

This code of conduct applies both within project spaces and in public spaces when an individual is representing the project or its community.

Instances of abusive, harassing, or otherwise unacceptable behavior can be reported by emailing `contact@gitlab.com`.

This Code of Conduct is adapted from the Contributor Covenant, version 1.1.0, available at http://contributor-covenant.org/version/1/1/0/.